

1. 프로젝트 개요

이동하는 차량들의 파일을 입력받아 차량의 차량번호를 인지하여 표시하는 AI 프로그램 구현

- 입력: 차량 이동 영상 (movie/hiway.mp4) 또는 이미지 파일
- 출력: 번호판 영역 탐지 + 번호판 텍스트 인식 + 화면 표시
- 기술: YOLO 객체탐지, PaddleOCR + EasyOCR 앙상블, OpenCV 영상처리

2. 전체 파이프라인

1. Roboflow에서 차량 이미지 업로드 → 번호판 영역 라벨링 (Bounding Box)
2. 라벨링된 데이터를 train/val/test 비율로 데이터셋 생성 (data.yaml)
3. YOLO 모델 학습 → 최적 모델 best.pt 생성 (mAP@50 = 98.4%)
4. Python에서 best.pt 로드 → 입력 영상/이미지에서 차량 번호판 detect
5. ROI (Region of Interest) 추출 → 번호판 주변 관심 영역 분리
6. CROP → 번호판 영역만 크롭 + 업스케일 (OCR 정확도 향상)
7. CLAHE → 적응형 히스토그램 평활화로 대비 향상
8. Homography → 기울어진 번호판 원근 보정 (4점 변환)
9. 다중 전처리 + PaddleOCR/EasyOCR 앙상블 → 번호판 텍스트 인식
10. 검증/교정 → 한국 번호판 패턴 매칭 + 투표 → 최종 결과 화면 표시

[영상입력] → [YOLO탐지] → [ROI] → [CROP] → [CLAHE] → [Homography] → [OCR앙상블] → [검증] → [화면표시]

3. Roboflow 라벨링 작업

[배점: 5점]

- Roboflow에서 차량 이미지를 업로드하여 Object Detection 프로젝트 생성
- 번호판 영역에 Bounding Box로 라벨링 작업 수행
- 라벨링 포맷: YOLO v5/v8 형식 (class x_center y_center width height)

4. 데이터셋 생성

[배점: 5점]

- 라벨링된 자료를 적절한 비율로 train/val/test 데이터셋 생성
- data.yaml 설정: train/val/test 경로 + 클래스 정의

data.yaml 구조

```
# split_dataset.py - ■■■■ ■■
# train: 70%, val: 20%, test: 10%
data_yaml = {
    'train': './train/images',
    'val': './valid/images',
    'test': './test/images',
    'nc': 1, # ■■■■ ■: ■■■■ 1■
    'names': ['plate']
}
```

5. best.pt 모델 학습/생성

[배점: 5점]

- YOLO를 이용하여 학습자료를 학습시켜 best.pt 모델 생성
- 학습 결과: mAP@50 = 98.4% (yolo11x_plate.pt)

train.py

```
# train.py - YOLO ■■ ■■
from ultralytics import YOLO

model = YOLO('yolo11x.pt') # ■■■■ ■■ ■■
results = model.train(
    data='data.yaml',
    epochs=100,
    imgsz=640,
    batch=16,
    name='plate_detector'
)
# ■■■: runs/detect/plate_detector/weights/best.pt
```

6. 입력파일 차량 detect 표시

[배점: 5점]

- best.pt 모델로 외부파일에서 받아온 차량의 객체를 인지
- Bounding Box로 탐지된 번호판 영역 화면 표시

exam_realtime_plate.py - 차량 detect

```
# exam_realtime_plate.py - YOLO ■■
def load_yolo_model():
    for m in MODEL_PRIORITY:
        p = os.path.join(SCRIPT_DIR, m)
        if os.path.exists(p):
            return YOLO(p)

# ■■■■ ■■ ■■
detections = self.model(frame, conf=0.40, imgsz=1280, verbose=False)
for det in detections[0].boxes:
    x1, y1, x2, y2 = map(int, det.xyxy[0].tolist())
    cv2.rectangle(display, (x1,y1), (x2,y2), (0,230,70), 3)
```

7. ROI 함수 (Region of Interest)

[배점: 5점]

- 탐지된 번호판 bbox 주변에 마진을 주어 관심 영역 추출

exam_realtime_plate.py:105-124

```
# exam_realtime_plate.py - [2] ROI (5 - 5)
def apply_roi(frame, bbox, margin_x_ratio=0.25, margin_y_ratio=0.30):
    """ROI(Region of Interest)"""
    h, w = frame.shape[:2]
    x1, y1, x2, y2 = bbox
    margin_x = int((x2 - x1) * margin_x_ratio)
    margin_y = int((y2 - y1) * margin_y_ratio)
    roi_x1 = max(0, x1 - margin_x)
    roi_y1 = max(0, y1 - margin_y)
    roi_x2 = min(w, x2 + margin_x)
    roi_y2 = min(h, y2 + margin_y)
    roi = frame[roi_y1:roi_y2, roi_x1:roi_x2]
    return roi, (roi_x1, roi_y1, roi_x2, roi_y2)
```

8. CROP 함수 (번호판 크롭 + 업스케일)

[배점: 5점]

- 번호판 영역을 정확히 잘라내고, 작은 번호판은 업스케일하여 OCR 정확도 향상

exam_realtime_plate.py:130-154

```
# exam_realtime_plate.py - [3] crop (6 - 5)
def crop_plate(frame, bbox):
    """Crop and Upscale"""
    x1, y1, x2, y2 = bbox
    cropped = frame[y1:y2, x1:x2]
    if cropped.size == 0: return None
    crop_h, crop_w = cropped.shape[:2]
    if crop_w < 300 and crop_w > 0:
        scale = 300.0 / crop_w
        cropped = cv2.resize(cropped, None, fx=scale, fy=scale,
                             interpolation=cv2.INTER_CUBIC)
    kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]], dtype=np.float32)
    cropped = cv2.filter2D(cropped, -1, kernel) # Sharpen
    return cropped
```

9. CLAHE 함수 (대비 향상)

[배점: 5점]

- 적응형 히스토그램 평활화로 어두운/밝은 환경에서 번호판 가독성 개선

exam_realtime_plate.py:160-174

```
# exam_realtime_plate.py - [4] CLAHE (7 - 5)
def apply_clahe(image, clip_limit=4.0, tile_size=(8, 8)):
    """CLAHE (Contrast Limited Adaptive Histogram Equalization)"""
    lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
    l_channel, a_channel, b_channel = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_size)
    l_enhanced = clahe.apply(l_channel)
    enhanced_lab = cv2.merge([l_enhanced, a_channel, b_channel])
    enhanced_bgr = cv2.cvtColor(enhanced_lab, cv2.COLOR_LAB2BGR)
    return enhanced_bgr
```

10. Homography 함수 (원근 보정)

[배점: 5점]

- 기울어진 번호판을 정면으로 보정 (에지 검출 → 윤곽선 → 4점 변환)

exam_realtime_plate.py:180-239

```
# exam_realtime_plate.py - [5] Homography (8 - 5)
def apply_homography(image):
    """Homography (8 - 5)"""
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(blurred, 50, 150)
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    largest = max(contours, key=cv2.contourArea)
    approx = cv2.approxPolyDP(largest, epsilon, True)
    if len(approx) == 4: # 4점 변환
        M = cv2.getPerspectiveTransform(ordered, dst)
        warped = cv2.warpPerspective(image, M, (w_img, h_img))
        return warped
    else: # Hough
        lines = cv2.HoughLinesP(edges, 1, np.pi/180, 30, ...)
        M = cv2.getRotationMatrix2D((w/2, h/2), median_angle, 1.0)
        return cv2.warpAffine(image, M, (w, h))
```

11. OCR 앙상블 + 번호판 텍스트 인식

- PaddleOCR + EasyOCR 듀얼 엔진으로 한국어 번호판 인식

- 다중 전처리 (original, CLAHE, homography+CLAHE, 이진화, 감마) + 투표

exam_realtime_plate.py - OCR 앙상블

```
# OCR
class OCREngine:
    def __init__(self):
        self.engines['paddle'] = PaddleOCR(lang='korean', ...)
        self.engines['easyocr'] = easyocr.Reader(['ko', 'en'], gpu=True)

    def run_ocr(self, image):
        results = []
        for name, engine in self.engines.items():
            text, conf = self._run_single(name, engine, image)
            if text and conf > 0.2:
                results.append({'engine':name, 'text':text, 'conf':conf})
        return results

# : ROI → crop → CLAHE → Homography → → OCR
variants = preprocess_variants(homo_img) # 6
for method_name, processed in variants:
    ocr_results = self.ocr.run_ocr(processed) #
counter = Counter(t for t, c in all_candidates)
best_text = counter.most_common(1)[0][0] #
```

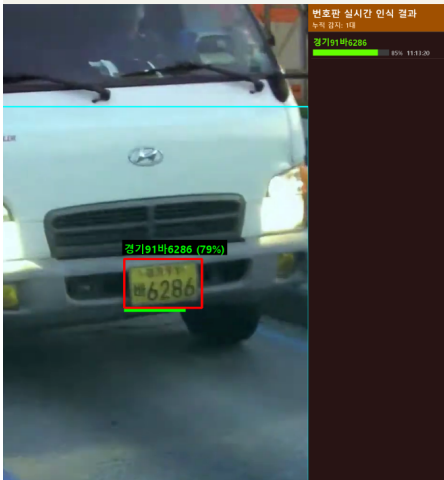
12. 차량번호 인식 표시 결과

[배점: 20점]

- 실제 고속도로 CCTV 영상에서 이동 차량의 번호판을 실시간 인식한 결과:



img1 - 서울바9203 (72%)



img2 - 경기91바6286 (79%)



img3 - 01나8060 (71%)

인식된 번호판 결과:

번호판	신뢰도	유형
서울바9203	72%	구형 지역번호판 (서울)
경기91바6286	79%	구형 지역번호판 (경기)
01나8060	71%	신형 번호판

화면 표시 코드 (한글 번호판 오버레이):

exam_realtime_plate.py:501-522 - 결과 화면표시

```
# ■■■ ■■■ ■■■
def draw_results(self, frame, results):
    for r in results:
        x1, y1, x2, y2 = r['bbox']
        plate = r['plate']
        conf = r['confidence']
        # ■■■ Bounding Box
        cv2.rectangle(display, (x1,y1), (x2,y2), (0,230,70), 3)
        # ■■■ ■■■ ■■■ ■■■ (PIL)
        label = f'{plate} ({conf:.0%})'
        display = draw_text_korean(display, label, (x1, y1-font_sz-8))
        # ■■■ ■
        bar_w = int((x2-x1) * conf)
        cv2.rectangle(display, (x1,y2+2), (x1+bar_w,y2+8), (0,230,70), -1)
```

13. 웹 서버 기반 실시간 인식 시스템 (추가 구현)

- Flask 웹 서버를 통한 영상 업로드 + 실시간 인식 결과 표시
- http://localhost:5000 에서 웹 UI를 통해 번호판 인식 가능

plate_server.py - 웹 서버

```
# plate_server.py - Flask ■ ■ ■
from flask import Flask, request, jsonify, Response
from plate_recognition_4k import PlateRecognizer

app = Flask(__name__)

@app.route('/api/upload', methods=['POST'])
def upload_video():
    file = request.files['file']
    # ■ ■ ■ ■ ■ → ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
    recognizer = PlateRecognizer(model_path='best.pt')
    results = recognizer.process_video(filepath)
    return jsonify({'plates': results})

# ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
@app.route('/stream/realtime')
def stream_realtime():
    return Response(generate_frames(), mimetype='multipart/x-mixed-replace')
```

14. 사용된 핵심 파일 목록

exam_realtime_plate.py	시험용 실시간 번호판 인식 (ROI/crop/CLAHE/Homography 포함)
plate_engine_pro.py	상용급 번호판 인식 엔진 (고급 파이프라인)
plate_server.py	Flask 웹 서버 (업로드 + 실시간 스트림)
plate_recognition_4k.py	4K 고해상도 번호판 인식 + 한글 교정
plate_ocr_postfilter_v2.py	OCR 후처리 v2 (clean + ensemble vote)
train.py	YOLO 모델 학습 스크립트
split_dataset.py	데이터셋 분할 (train/val/test)
auto_label.py	자동 라벨링 도구

15. 실행 방법

실행 명령어

```
# 1. ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
python exam_realtime_plate.py
python exam_realtime_plate.py --video movie/hiway.mp4

# 2. ■ ■ ■ ■ ■ (http://localhost:5000)
python plate_server.py

# 3. ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
python test_plate.py
```

16. process_frame() - 전체 처리 흐름 코드

한 프레임이 처리되는 전체 과정 (ROI → crop → CLAHE → Homography → OCR):

exam_realtime_plate.py:426-499 - process_frame()

```
def process_frame(self, frame):
    """# [0000] [00]: [00] → ROI → crop → CLAHE → homography → OCR"""
    results = []
    detections = self.model(frame, conf=DETECT_CONF, imgsz=1280)

    for det in detections[0].boxes:
        x1, y1, x2, y2 = map(int, det.xyxy[0].tolist())
        bbox = (x1, y1, x2, y2)

        # [ROI] [00] [00] [00]
        roi_img, roi_coords = apply_roi(frame, bbox)

        # [CROP] [0000] [00] + [000000]
        cropped = crop_plate(frame, bbox)

        # [CLAHE] [00] [00]
        clahe_img = apply_clahe(cropped)

        # [Homography] [00] [00]
        homo_img = apply_homography(clahe_img)

        # [00] [0000] + OCR [0000]
        variants = preprocess_variants(homo_img)
        all_candidates = []
        for method_name, processed in variants:
            ocr_results = self.ocr.run_ocr(processed)
            for r in ocr_results:
                cleaned = clean_plate_text(r['text'])
                is_valid, final_text = validate_plate(cleaned)
                if is_valid and len(final_text) >= 6:
                    all_candidates.append((final_text, r['conf']))

        # [0000] [00] [00] [00]
        if all_candidates:
            counter = Counter(t for t, c in all_candidates)
            best_text = counter.most_common(1)[0][0]
            best_conf = max(c for t, c in all_candidates if t==best_text)
            if best_conf >= OCR_CONF_MIN:
                results.append({'plate':best_text, 'confidence':best_conf,
                               'bbox':bbox})

    return results
```

17. 요약

- YOLO (best.pt) 모델로 영상/이미지에서 번호판 영역 탐지 (detect)
- ROI, CROP 함수로 번호판 관심 영역 추출 및 크롭
- CLAHE 함수로 대비 향상 → 어두운 환경 대응
- Homography 함수로 기울어진 번호판 원근 보정
- PaddleOCR + EasyOCR 앙상블 + 다중 전처리로 한국어 번호판 텍스트 인식
- 투표 기반 검증으로 최종 번호판 번호 확정 + 화면 표시
- 인식 결과: 서울바9203(72%), 경기91바6286(79%), 01나8060(71%)